

Aula 03

Assincronia

quando o código precisa esperar

Quase tudo é instantâneo

```
const soma = 2 + 3;  
const nome = "ana".toUpperCase();
```

Mas algumas coisas não são

Operação	Tempo
Somar dois números	0,000001 ms
Ler arquivo do disco	5 ms
Consultar banco	20 ms
Chamar API na internet	300 ms

Por que 300 ms é um problema?

JavaScript executa **uma coisa de cada vez**.

Se o programa parasse 300 ms a cada requisição:

3 usuários por segundo

Inaceitável para um servidor.



O garçom síncrono

1. Anota seu pedido
2. Vai pra cozinha
3. **Fica parado olhando o fogão**
4. Traz seu prato
5. Só então atende a próxima mesa

Uma mesa por vez 



O garçom assíncrono

1. Anota seu pedido
2. Entrega na cozinha
3. **Vai atender outras mesas**
4. Cozinha grita "pedido 5 pronto!"
5. Ele busca e entrega

Dez mesas ao mesmo tempo 🚀

← **Este é o Node.js**

Detalhe importante

O Node **não faz duas coisas ao mesmo tempo.**

Ele apenas **nunca fica parado esperando.**

Enquanto o disco ou a rede trabalham,
ele executa outro pedaço de código.

A assincronia acontecendo

```
setTimeout(() => {  
  console.log("passaram 2 segundos");  
}, 2000);  
  
console.log("esta linha aparece PRIMEIRO");
```

Saída:

```
esta linha aparece PRIMEIRO  
passaram 2 segundos
```



setTimeout(funcao, ms)

O que faz: executa a função depois de N milissegundos

Quando usar: adiar algo, simular demora,
esperar o usuário parar de digitar (aula 09)

```
setTimeout(() => console.log("oi"), 1000);    // 1 segundo  
  
const timer = setTimeout(fn, 400);  
clearTimeout(timer);    // cancela antes de disparar
```

⚠ 1 segundo = 1000 ms

O jeito antigo: callbacks

```
buscarUsuario(1, (usuario) => {  
  buscarPedidos(usuario.id, (pedidos) => {  
    buscarItens(pedidos[0].id, (itens) => {  
      calcularFrete(itens, (frete) => {  
        console.log(frete);    // 🤪  
      });  
    });  
  });  
});
```

"Callback hell" — a pirâmide da perdição

Promise: um vale para o futuro

Você pede um lanche → recebe uma **senha**

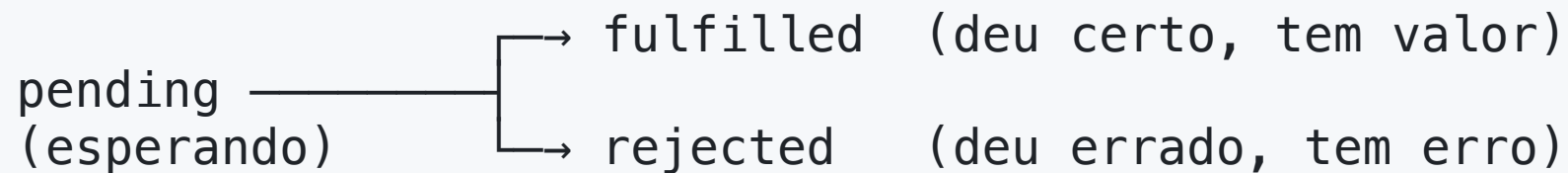
A senha **não é** o lanche.

É a promessa de que ele virá.

Você guarda no bolso e **continua sua vida**.

Depois troca a senha pelo lanche —
ou pela notícia de que o pão acabou.

Os 3 estados de uma Promise



Uma vez resolvida, **nunca mais muda**.

```
const p = fetch(url);  
console.log(p);    // Promise { <pending> }
```

Consumindo com `.then()`

```
fetch("https://viacep.com.br/ws/01001000/json/")  
  .then((resposta) => resposta.json())  
  .then((dados) => console.log(dados.logradouro))  
  .catch((erro) => console.log("deu ruim:", erro));
```

Melhor que callbacks: encadeia em vez de aninhar.

Mas ainda não é o ideal...

async / await ★

```
async function buscarCep(cep) {  
  const resposta = await fetch(`.../${cep}/json/`);  
  const dados = await resposta.json();  
  return dados;  
}
```

Parece código normal, de cima para baixo

Só duas regras

await

"espere esta Promise e me dê o valor de dentro"

async

"esta função tem **await** e devolve uma Promise"

O ERRO Nº 1 DO MÓDULO

```
const dados = fetch(url);    // ✗  
const dados = await fetch(url); // ✓
```

Viu isso no console?

```
Promise { <pending> }
```

Faltou um **await**.

Função `async` sempre devolve Promise

```
async function pegarNumero() {  
  return 42;           // devolvo um número...  
}  
  
pegarNumero();          // Promise { 42 } ← sai Promise  
await pegarNumero();    // 42
```

Quem chama uma função `async`
também precisa dar `await`.



fetch()

`fetch()` — a ficha técnica

O que faz: faz uma requisição HTTP

Devolve: uma Promise com a **RESPOSTA**

(não com os dados!)

```
const resposta = await fetch(url);    // 1. a resposta
const dados = await resposta.json();  // 2. os dados
//           ↑ TAMBÉM é assíncrono!
```

São dois `await`. Muita gente esquece o segundo.

O que vem na resposta

Propriedade	O que é
<code>resposta.ok</code>	<code>true</code> se o status for 2xx
<code>resposta.status</code>	o número: 200, 404, 500...
<code>resposta.json()</code>	converte o corpo (assíncrono)
<code>resposta.text()</code>	o corpo como texto puro

Tratando erros

```
try {  
  const resposta = await fetch(url);  
  if (!resposta.ok) {  
    throw new Error(`Status ${resposta.status}`);  
  }  
  return await resposta.json();  
} catch (erro) {  
  console.error("Falhou:", erro.message);  
  return null;  
} finally {  
  console.log("encerrado");  
}
```

throw e o objeto erro

```
throw new Error("mensagem aqui");  
//      ↑ interrompe tudo e pula para o catch
```

No `catch`, você recebe o objeto de erro:

Propriedade	O que tem
<code>erro.message</code>	a mensagem legível ★
<code>erro.name</code>	o tipo do erro
<code>erro.stack</code>	onde aconteceu (para você, não para o usuário)
<code>erro.code</code>	código do sistema (ex.: <code>"ENOENT"</code>)

! Pegadinha do `fetch`

O `fetch` **não** lança erro em 404 ou 500.

Ele só falha se a **rede** cair.

```
if (!resposta.ok) {  
  throw new Error(`Status ${resposta.status}`);  
}
```

Você precisa checar **na mão**.

Três coisas para guardar

<code>try</code>	o código que pode dar errado
<code>catch (erro)</code>	o que fazer se der (<code>erro.message</code>)
<code>finally</code>	roda sempre — deu certo ou não

Esperando 3 CEPs — jeito lento

```
const a = await buscarCep("01001000"); // 300ms  
const b = await buscarCep("20040030"); // 300ms  
const c = await buscarCep("30130010"); // 300ms
```

Total: 900 ms

Mas eles são independentes... por que a fila?

Promise.all — jeito rápido

```
const [a, b, c] = await Promise.all([
  buscarCep("01001000"),
  buscarCep("20040030"),
  buscarCep("30130010"),
]);
```

Total: ~300 ms

As três saem **juntas**.

Quando usar cada um

Situação	Use
B precisa do resultado de A	<code>await</code> em sequência
A, B e C são independentes	<code>Promise.all</code>
Quero o resultado mesmo com falhas	<code>Promise.allSettled</code>

⚠ `Promise.all` é tudo-ou-nada: uma falha derruba todas.

⚠ A armadilha do `forEach`

```
// ❌ o forEach não espera NADA
ceps.forEach(async (cep) => {
  const dados = await buscarCep(cep);
});
console.log("acabou"); // imprime antes de tudo

// ✅
for (const cep of ceps) {
  const dados = await buscarCep(cep);
}
```

Medindo o tempo

```
const inicio = performance.now();  
// ... algo demorado ...  
console.log(`${Math.round(performance.now() - inicio)} ms`);
```

Ou, mais simples:

```
console.time("busca");  
// ... algo demorado ...  
console.timeEnd("busca");    // busca: 487ms
```

Lendo e escrevendo arquivos

node:fs/promises

```
import { readFile, writeFile, appendFile }  
  from "node:fs/promises";
```

Função	O que faz
<code>readFile(arq, "utf-8")</code>	lê o conteúdo
<code>writeFile(arq, texto)</code>	substitui o arquivo inteiro
<code>appendFile(arq, texto)</code>	acrescenta ao final

`node:` = módulo interno do Node, não do npm

⚠️ Dois detalhes do `readFile`

```
await readFile("dados.txt");           // Buffer <48 65 6c...> 🙄  
await readFile("dados.txt", "utf-8");  // "Olá mundo" ✅
```

Sem `"utf-8"`, volta em **bytes crus**.

E lembre: `writeFile` **substitui tudo**.

Para acrescentar, é `appendFile`.

O par JSON ↔ arquivo

```
// ler
const texto = await readFile("dados.json", "utf-8");
const dados = JSON.parse(texto);

// escrever
await writeFile(
  "dados.json",
  JSON.stringify(dados, null, 2) // ← 2 = indentação legível
);
```

`erro.code` — erros do sistema

```
try {  
  await readFile("dados.json", "utf-8");  
} catch (erro) {  
  if (erro.code === "ENOENT") {  
    return []; // arquivo ainda não existe: normal!  
  }  
  throw erro; // qualquer outro: repassa  
}
```

`ENOENT` = **Error NO ENTry**

→ "esse arquivo não existe"

Recapitulando

1. O Node **delega e continua** — nunca fica parado esperando
2. `await` desembrulha a Promise · sem ele: `Promise { <pending> }`
3. Independentes → `Promise.all` · tudo que pode falhar → `try/catch`

Mão na massa 🚀

`aula-03/exercicios/README.md`

Próxima aula: vamos criar um servidor 🖥️